1). Watch the Terraform-06 video.

Completed

2). Execute the script shown in the video.

Implementation Steps

1. Created a project directory named terraform-06.
2. Created two directories named aws_instance and dev.
3. Inside the aws_instance directory, created a main.tf file.
4. Defined an input variable named ami to receive the Amazon Machine Image ID.
5. Created an EC2 resource using the aws_instance resource block.
6. Inside the dev directory, created another main.tf file.
7. Configured the AWS provider with the region us-east-1.
8. Called the EC2 module using the module block and provided the AMI value.
9. Executed terraform init to initialize the configuration.
10. Executed terraform validate to validate the configuration.
11. Executed terraform plan to review the resources.
12. Executed terraform apply to create the EC2 instance.
13. Verified that the EC2 instance was successfully created in the AWS Management Console.
14. Executed terraform destroy to remove the EC2 instance after completing the practice.

Project Folder Structure

terraform-06/

```
│
├── aws_instance/
│   └── main.tf
│
└── dev/
    └── main.tf
```

Code 1: EC2 Module

File: aws_instance/main.tf

```
variable "ami" {
  type = string
}

resource "aws_instance" "webserver" {
  ami         = var.ami
  instance_type = "t3.micro"
}
```

Code 2: Root Configuration

File: dev/main.tf

```
provider "aws" {
  region = "us-east-1"
}

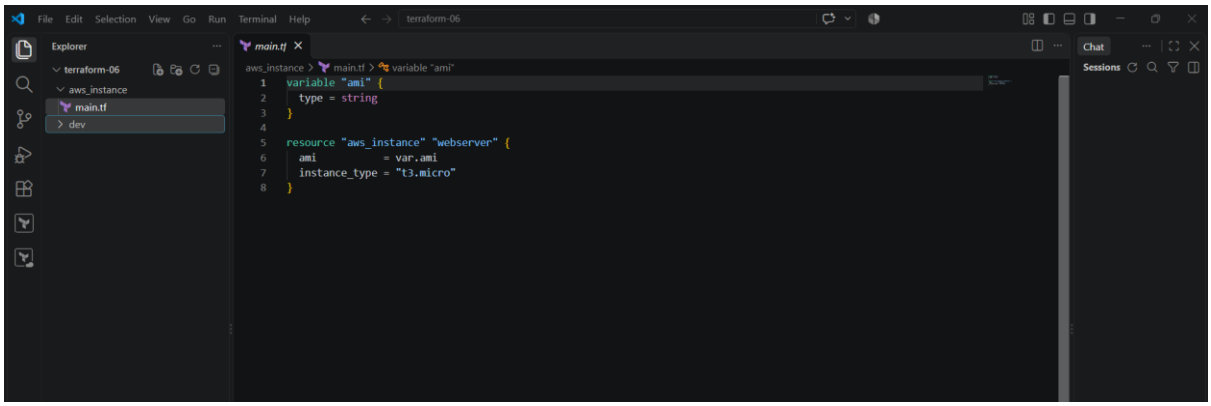
module "dev_webserver" {
  source = "../aws_instance"
  ami    = "ami-02f3f602d23f1659d"
}
```

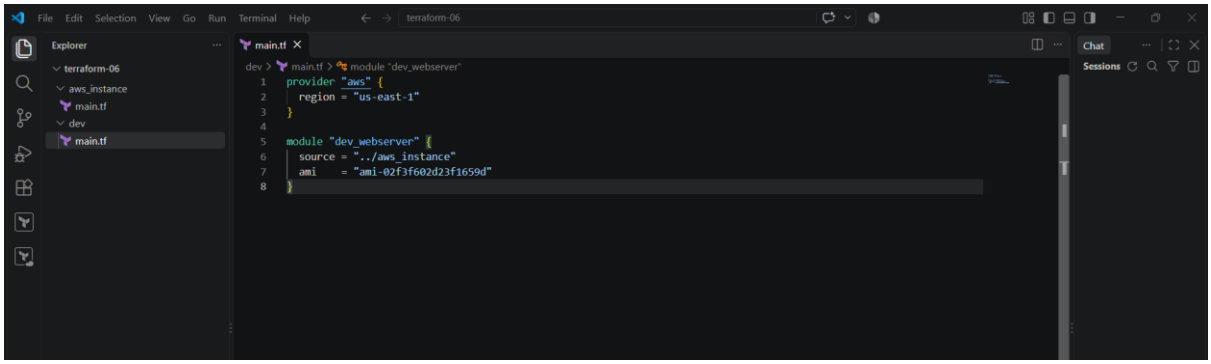
Commands Executed

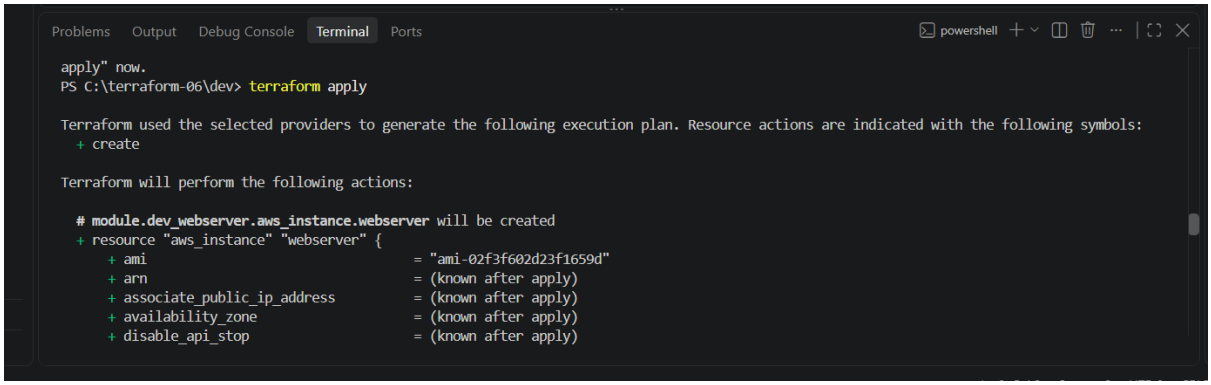
```
terraform init
terraform validate
terraform plan
terraform apply
terraform destroy
```

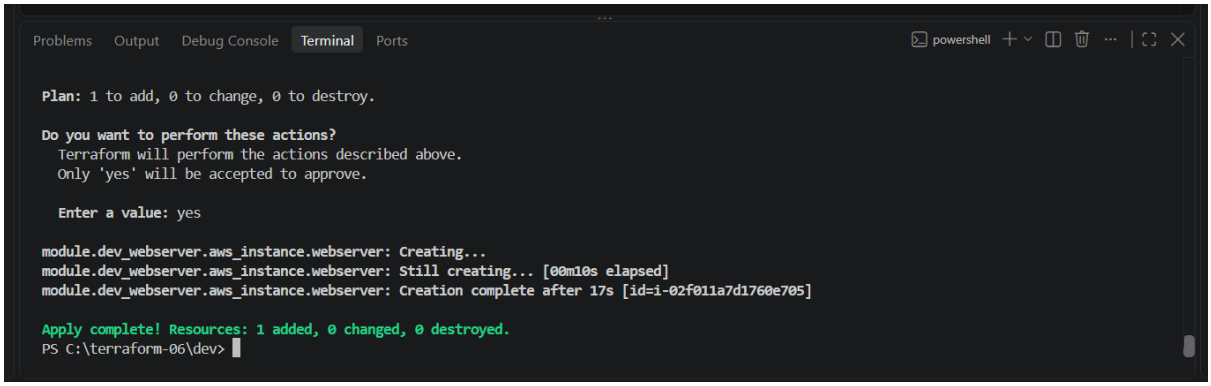
Result

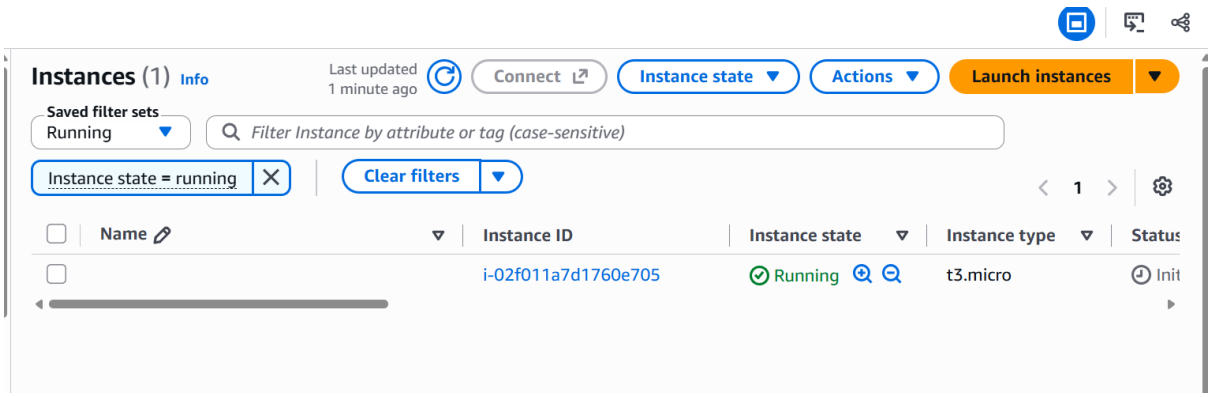
Successfully created an EC2 instance using a reusable Terraform module and verified the instance in the AWS Console. Destroyed the instance after completing the practice.











3). Provision EC2, S3, and VPC using Terraform modules.

Implementation Steps

1. Installed Terraform on the local machine and configured AWS credentials.
2. Created a project directory named terraform-modules.
3. Created the root configuration files: main.tf, variables.tf, and outputs.tf.
4. Created a modules directory inside the root project directory.
5. Created three separate module directories named aws_vpc, aws_ec2, and aws_s3.
6. Created the VPC module to provision a VPC and public subnet.
7. Configured an Internet Gateway, Route Table, and Route Table Association for internet connectivity.
8. Created the EC2 module to provision an EC2 instance inside the subnet created by the VPC module.
9. Configured a Security Group to allow inbound HTTP traffic on port 80.

10. Configured user data to install and start the Apache web server on the EC2 instance.
11. Created the S3 module to provision an S3 bucket with public access blocked.
12. Created output variables to retrieve the VPC ID, subnet ID, EC2 instance ID, public IP, and S3 bucket name.
13. Called all three modules from the root main.tf file and passed the required values between modules.
14. Executed terraform init to initialize Terraform and download the AWS provider.
15. Executed terraform validate to validate the configuration.
16. Executed terraform plan to review the resources that would be created.
17. Executed terraform apply and confirmed the deployment.
18. Successfully provisioned 9 AWS resources using Terraform modules.
19. Verified the VPC, subnet, EC2 instance, Security Group, and S3 bucket in the AWS Management Console.
20. Accessed the Apache web page using the EC2 public IP address.

Project Folder Structure

```
terraform-modules/
│
├── main.tf
├── variables.tf
├── outputs.tf
│
└── modules/
    │
    ├── aws_vpc/
    │   ├── main.tf
    │   └── outputs.tf
    │
    ├── aws_ec2/
```

```
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
│
└── aws_s3/
    ├── main.tf
    └── outputs.tf
```

Code 1: VPC Module

File: modules/aws_vpc/main.tf

```
resource "aws_vpc" "main" {
  cidr_block       = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-vpc"
  }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true

  tags = {
    Name = "terraform-public-subnet"
  }
}
```

```
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "terraform-igw"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "terraform-public-route-table"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

File: modules/aws_vpc/outputs.tf

```
output "vpc_id" {
```

```
 value = aws_vpc.main.id
}

output "subnet_id" {
 value = aws_subnet.public.id
}
```

Code 2: EC2 Module

File: modules/aws_ec2/main.tf

```
data "aws_ami" "amazon_linux" {
 most_recent = true
 owners      = ["amazon"]

 filter {
  name   = "name"
  values = ["al2023-ami-*-x86_64"]
 }

 filter {
  name   = "architecture"
  values = ["x86_64"]
 }
}

resource "aws_security_group" "web" {
 name   = "terraform-module-http-sg"
 vpc_id = var.vpc_id

 ingress {
```

```
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }
}

resource "aws_instance" "web" {
 ami                    = data.aws_ami.amazon_linux.id
 instance_type          = var.instance_type
 subnet_id              = var.subnet_id
 vpc_security_group_ids = [aws_security_group.web.id]
 associate_public_ip_address = true

 user_data = <<-EOF
  #!/bin/bash
  dnf install -y httpd
  systemctl enable --now httpd
  echo "<h1>EC2 Created Using Terraform Modules</h1>" > /var/www/html/index.html
 EOF
```

```
  tags = {
    Name = var.instance_name
  }
}
```

File: modules/aws_ec2/variables.tf

```
variable "vpc_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}

variable "instance_name" {
  type = string
}
```

File: modules/aws_ec2/outputs.tf

```
output "instance_id" {
  value = aws_instance.web.id
}

output "public_ip" {
  value = aws_instance.web.public_ip
```

```
}
```

Code 3: S3 Module

File: modules/aws_s3/main.tf

```
variable "bucket_name" {
  type = string
}

resource "aws_s3_bucket" "main" {
  bucket = var.bucket_name

  tags = {
    Name = "terraform-module-bucket"
  }
}

resource "aws_s3_bucket_public_access_block" "main" {
  bucket = aws_s3_bucket.main.id

  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```

File: modules/aws_s3/outputs.tf

```
output "s3_bucket_name" {
  value = aws_s3_bucket.main.bucket
}
```

Code 4: Root Configuration

File: main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.70"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

module "vpc" {
  source = "./modules/aws_vpc"
}

module "ec2" {
  source = "./modules/aws_ec2"

  vpc_id       = module.vpc.vpc_id
  subnet_id    = module.vpc.subnet_id
  instance_type = "t3.micro"
  instance_name = "module-web-server"
}

module "s3" {
```

```
  source     = "./modules/aws_s3"
  bucket_name = var.bucket_name
}
```

File: variables.tf

```
variable "bucket_name" {
  description = "Globally unique S3 bucket name"
  type       = string
}
```

File: outputs.tf

```
output "vpc_id" {
  value = module.vpc.vpc_id
}

output "ec2_instance_id" {
  value = module.ec2.instance_id
}

output "ec2_public_ip" {
  value = module.ec2.public_ip
}

output "s3_bucket_name" {
  value = module.s3.s3_bucket_name
}
```

Commands Executed

Provided a globally unique S3 bucket name:

```
terraform init
terraform validate
```

terraform plan -var="bucket_name=terraform-06-modules-poojith-20261101-956035085469"

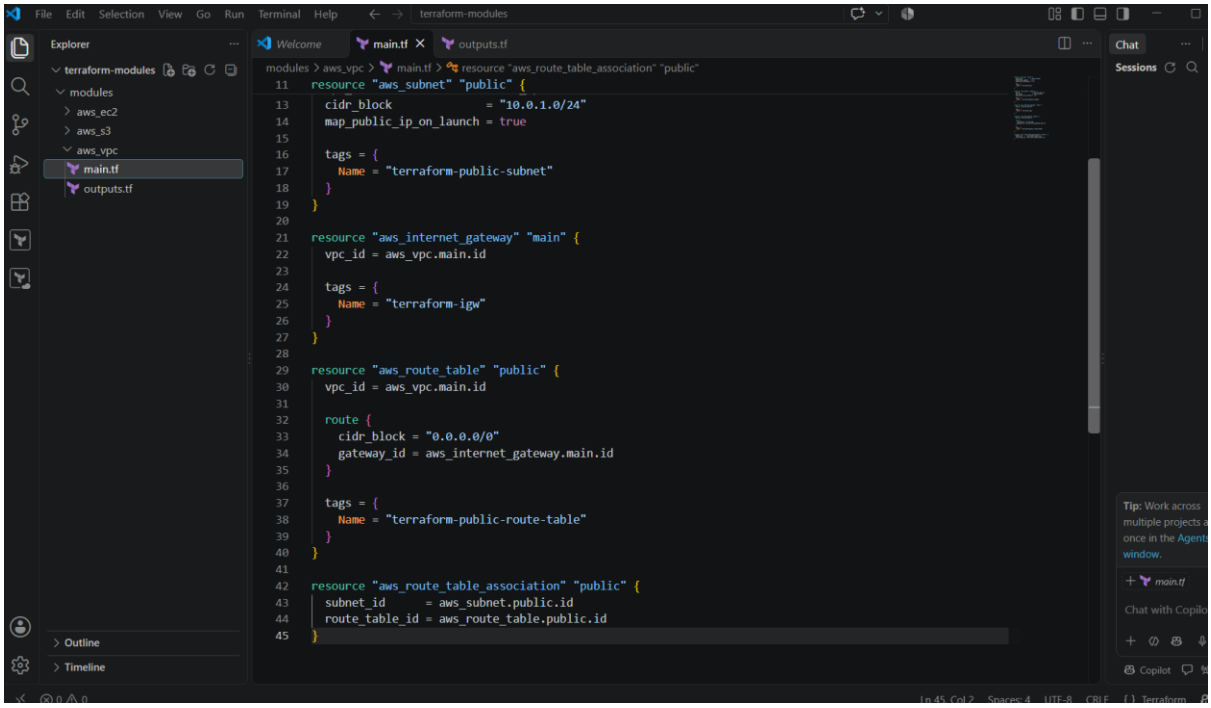
terraform apply -var="bucket_name=terraform-06-modules-poojith-20261101-956035085469"

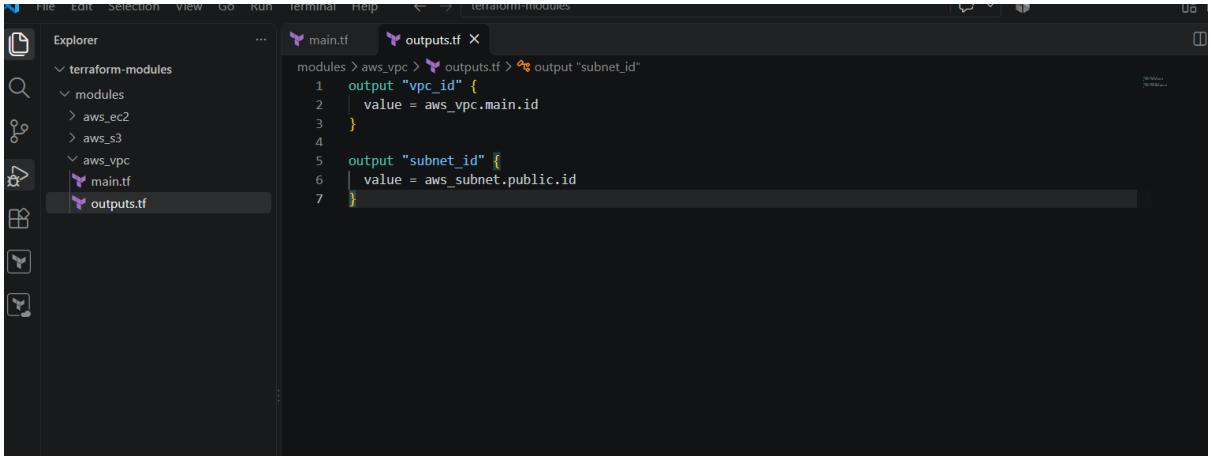
Result

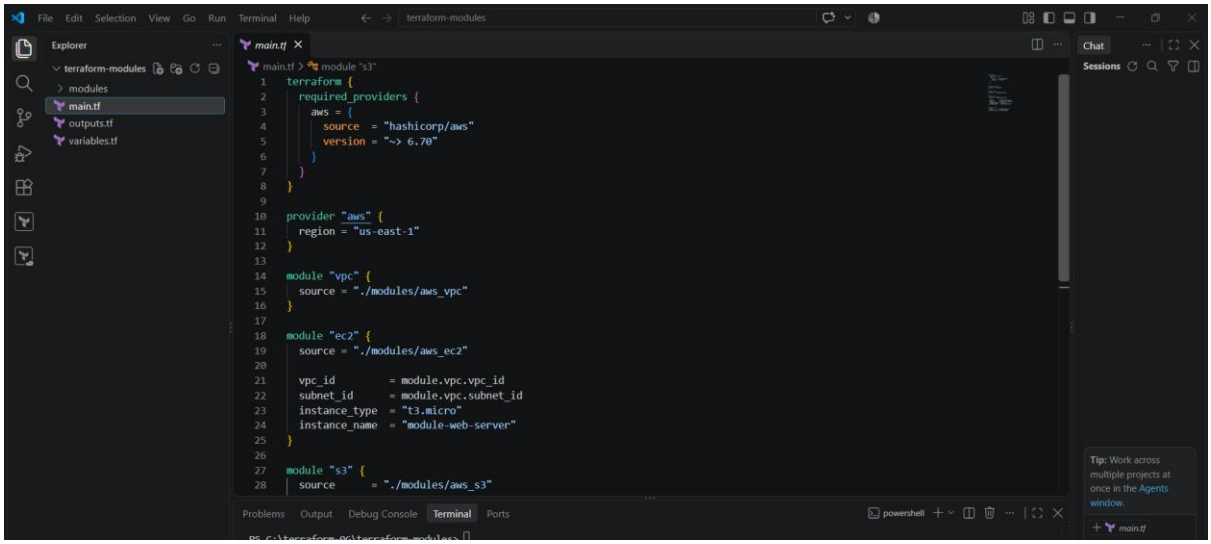
Successfully provisioned 9 AWS resources using reusable Terraform modules.

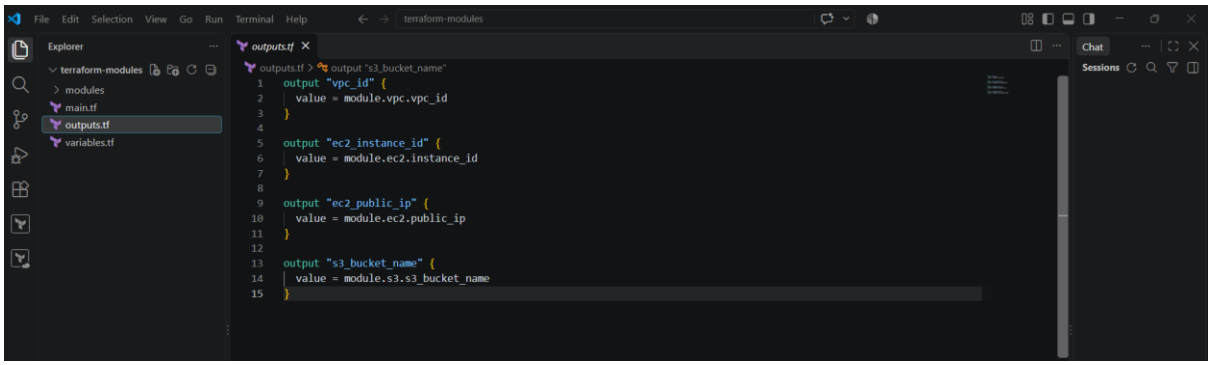
The resources included:

- 1 VPC
- 1 Public Subnet
- 1 Internet Gateway
- 1 Route Table
- 1 Route Table Association
- 1 Security Group
- 1 EC2 Instance
- 1 S3 Bucket
- 1 S3 Public Access Block configuration

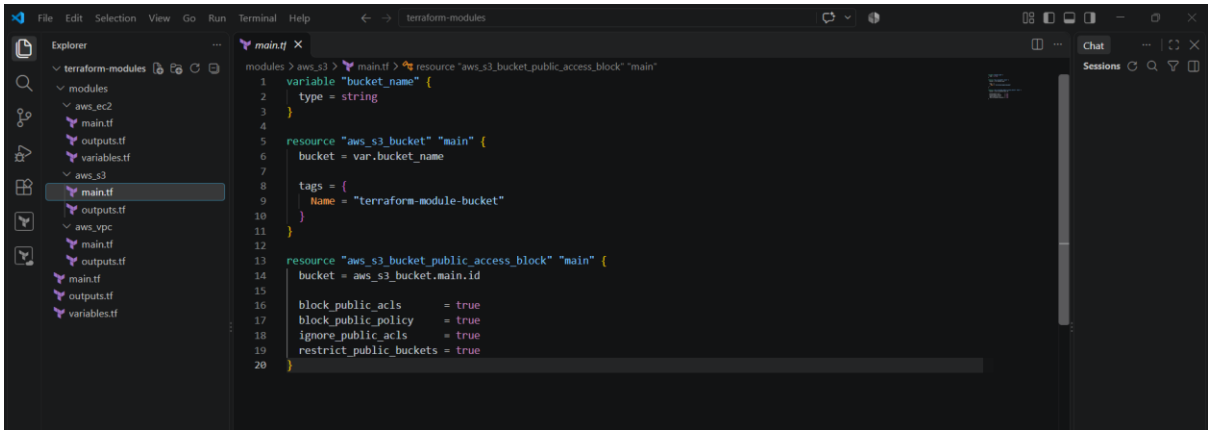


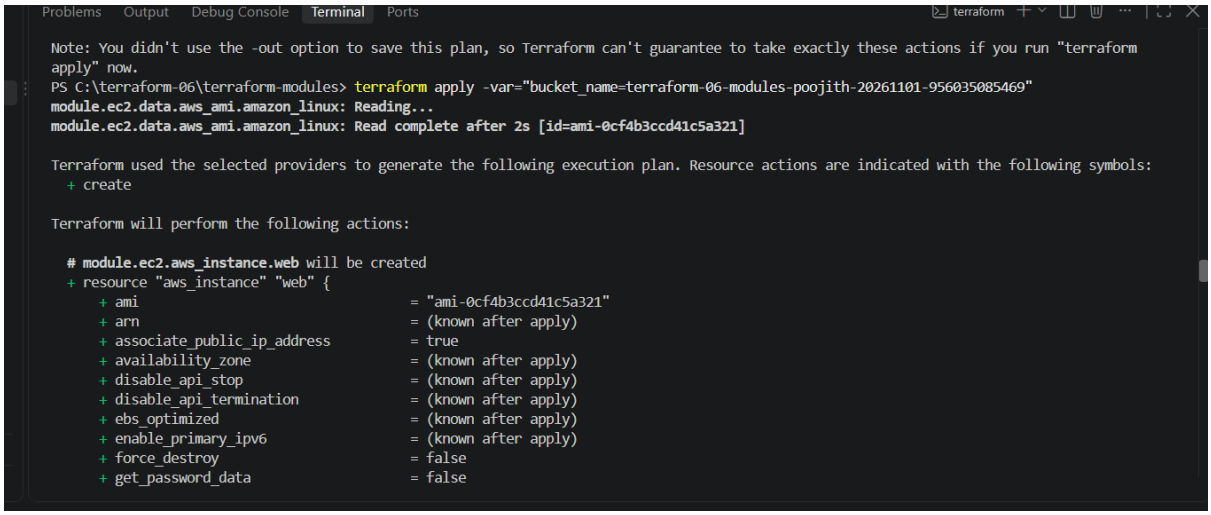

Explorer
terraform-modules
modules
aws_ec2
aws_s3
aws_vpc
main.tf
outputs.tf
main.tf
outputs.tf
modules > aws_vpc > outputs.tf > output "subnet_id"
output "vpc_id" {
value = aws_vpc.main.id
}
output "subnet_id" {
value = aws_subnet.public.id
}

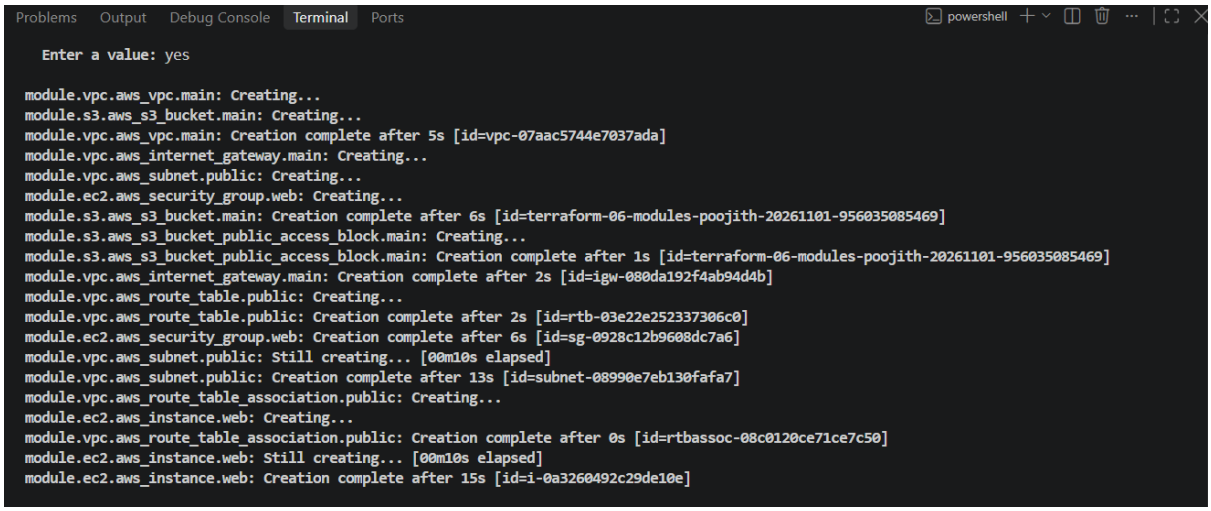

Explorer
terraform-modules
modules
main.tf
outputs.tf
variables.tf
main.tf
main.tf > module "s3"
terraform {
required_providers {
aws = {
source = "hashicorp/aws"
version = "~> 6.70"
}
}
}
provider "aws" {
region = "us-east-1"
}
module "vpc" {
source = "./modules/aws_vpc"
}
module "ec2" {
source = "./modules/aws_ec2"
vpc_id = module.vpc.vpc_id
subnet_id = module.vpc.subnet_id
instance_type = "t3.micro"
instance_name = "module-web-server"
}
module "s3" {
source = "./modules/aws_s3"
Chat
Sessions
Tip: Work across multiple projects at once in the Agents window.
Problems
Output
Debug Console
Terminal
Ports
powershell


Explorer
terraform-modules
modules
main.tf
outputs.tf
variables.tf
outputs.tf
outputs.tf > output "s3_bucket_name"
output "vpc_id" {
value = module.vpc.vpc_id
}
output "ec2_instance_id" {
value = module.ec2.instance_id
}
output "ec2_public_ip" {
value = module.ec2.public_ip
}
output "s3_bucket_name" {
value = module.s3.s3_bucket_name
}
Chat
Sessions


Explorer
terraform-modules
modules
main.tf
outputs.tf
variables.tf
variables.tf
variables.tf > variable "bucket_name"
variable "bucket_name" {
description = "Globally unique S3 bucket name"
type = string
}
Chat
Sessions


File Edit Selection View Go Run Terminal Help
terraform-modules
Explorer
terraform-modules
modules
aws_ec2
main.tf
outputs.tf
variables.tf
aws_s3
main.tf
outputs.tf
aws_vpc
main.tf
outputs.tf
main.tf
outputs.tf
variables.tf
main.tf
modules > aws_s3 > main.tf > resource "aws_s3_bucket_public_access_block" "main"
variable "bucket_name" {
type = string
}
resource "aws_s3_bucket" "main" {
bucket = var.bucket_name
tags = {
Name = "terraform-module-bucket"
}
}
resource "aws_s3_bucket_public_access_block" "main" {
bucket = aws_s3_bucket.main.id
block_public_acls = true
block_public_policy = true
ignore_public_acls = true
restrict_public_buckets = true
}
Chat
Sessions

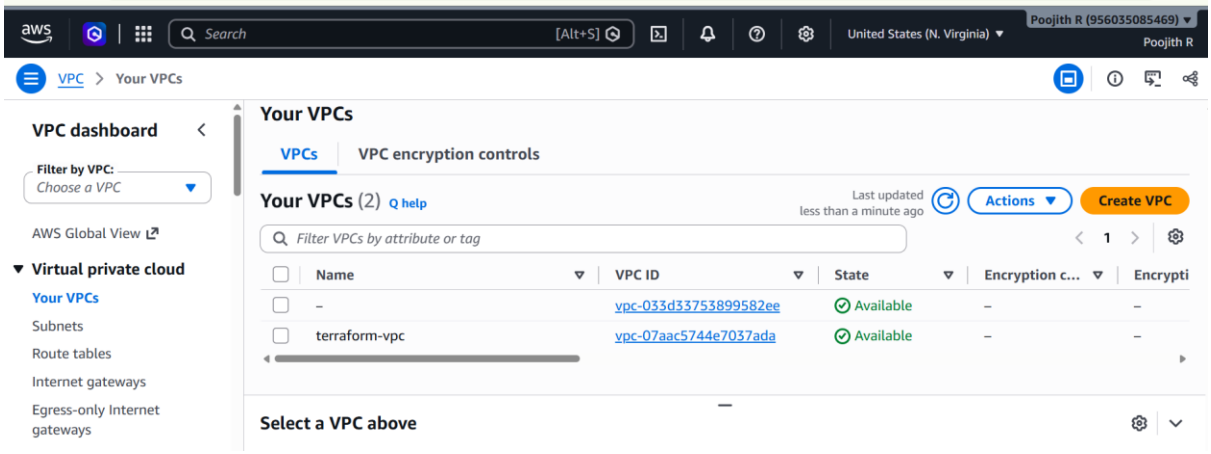

Problems Output Debug Console Terminal Ports
terraform
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\terraform-06\terraform-modules> terraform apply -var="bucket_name=terraform-06-modules-poojith-20261101-956035085469"
module.ec2.data.aws_ami.amazon_linux: Reading...
module.ec2.data.aws_ami.amazon_linux: Read complete after 2s [id=ami-0cf4b3ccd41c5a321]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
module.ec2.aws_instance.web will be created
+ resource "aws_instance" "web" {
+ ami = "ami-0cf4b3ccd41c5a321"
+ arn = (known after apply)
+ associate_public_ip_address = true
+ availability_zone = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ enable_primary_ipv6 = (known after apply)
+ force_destroy = false
+ get_password_data = false

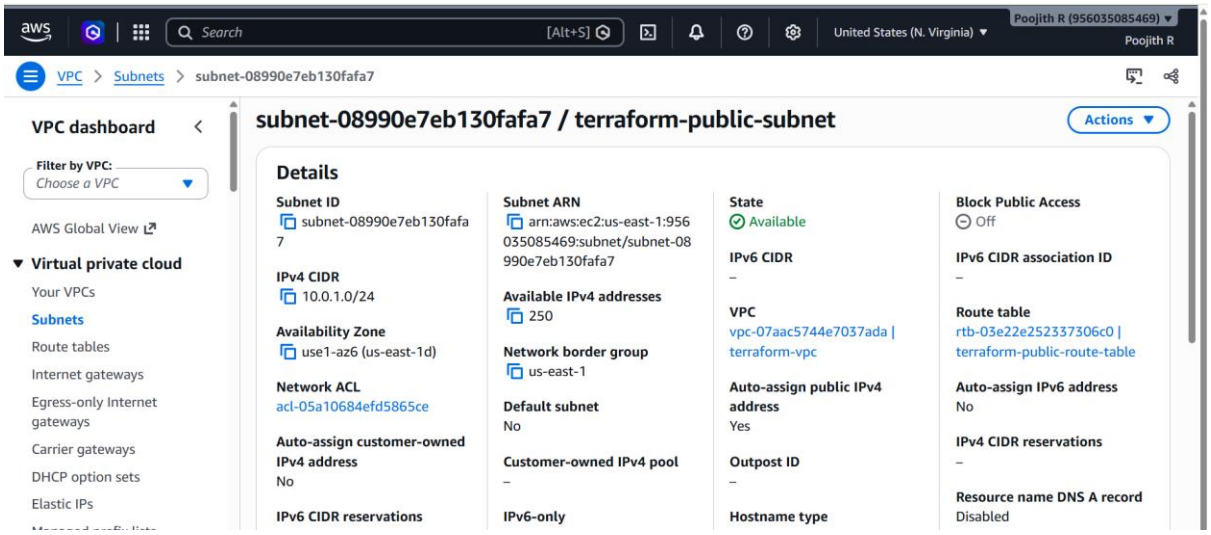

Problems Output Debug Console Terminal Ports
powershell
Enter a value: yes
module.vpc.aws_vpc.main: Creating...
module.s3.aws_s3_bucket.main: Creating...
module.vpc.aws_vpc.main: Creation complete after 5s [id=vpc-07aac5744e7037ada]
module.vpc.aws_internet_gateway.main: Creating...
module.vpc.aws_subnet.public: Creating...
module.ec2.aws_security_group.web: Creating...
module.s3.aws_s3_bucket.main: Creation complete after 6s [id=terraform-06-modules-poojith-20261101-956035085469]
module.s3.aws_s3_bucket_public_access_block.main: Creating...
module.s3.aws_s3_bucket_public_access_block.main: Creation complete after 1s [id=terraform-06-modules-poojith-20261101-956035085469]
module.vpc.aws_internet_gateway.main: Creation complete after 2s [id=igw-080da192f4ab94d4b]
module.vpc.aws_route_table.public: Creating...
module.vpc.aws_route_table.public: Creation complete after 2s [id=rtb-03e22e252337306c0]
module.ec2.aws_security_group.web: Creation complete after 6s [id=sg-0928c12b9608dc7a6]
module.vpc.aws_subnet.public: Still creating... [00m10s elapsed]
module.vpc.aws_subnet.public: Creation complete after 13s [id=subnet-08990e7eb130fafa7]
module.vpc.aws_route_table_association.public: Creating...
module.ec2.aws_instance.web: Creating...
module.vpc.aws_route_table_association.public: Creation complete after 0s [id=rtbassoc-08c0120ce71ce7c50]
module.ec2.aws_instance.web: Still creating... [00m10s elapsed]
module.ec2.aws_instance.web: Creation complete after 15s [id=i-0a3260492c29de10e]

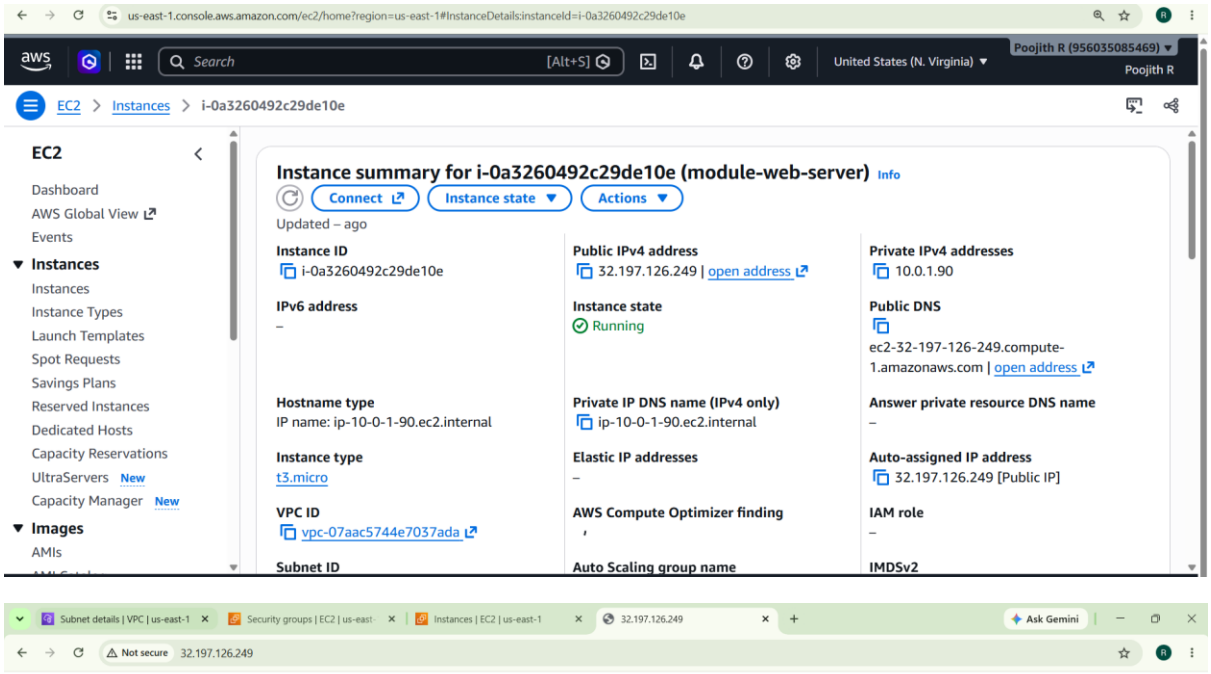
```
Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

Outputs:

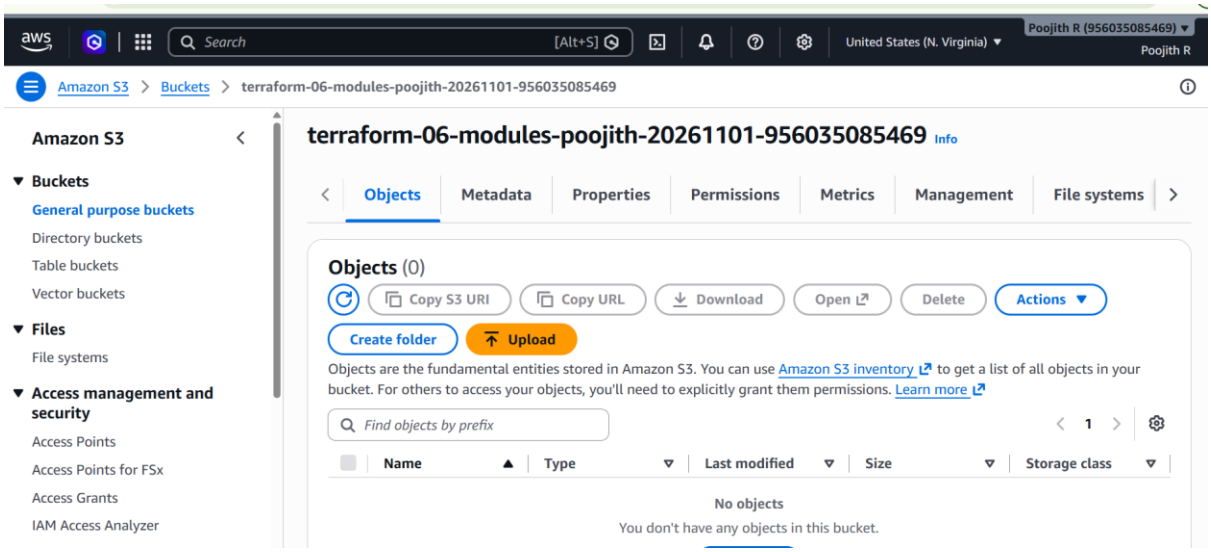
ec2_instance_id = "i-0a3260492c29de10e"
ec2_public_ip = "32.197.126.249"
s3_bucket_name = "terraform-06-modules-poojith-20261101-956035085469"
vpc_id = "vpc-07aac5744e7037ada"
PS C:\terraform-06\terraform-modules>
```







EC2 Created Using Terraform Modules



4). Provision EC2 for 3 different environments (Dev, Staging, and Prod) using Terraform workspaces.

Implementation Steps

1. Created a project directory named terraform_workspace.
2. Created two Terraform configuration files named main.tf and variables.tf.
3. Configured the AWS provider with the region us-east-1.
4. Created an EC2 resource named appserver in the main.tf file.
5. Used terraform.workspace to select the AMI ID based on the currently selected workspace.
6. Configured the instance name using the workspace-specific tag variable.
7. Defined AMI IDs for Dev, Staging, and Prod environments in variables.tf.
8. Defined separate instance names for all three environments.
9. Executed terraform init to initialize the Terraform configuration.
10. Created three workspaces using terraform workspace new Dev, terraform workspace new Staging, and terraform workspace new Prod.
11. Executed terraform workspace list to verify the created workspaces.

Dev Environment:

12. Selected the Dev workspace using terraform workspace select Dev.
13. Executed terraform plan to review the execution plan.
14. Executed terraform apply and successfully created the Dev EC2 instance.

Staging Environment:

15. Selected the Staging workspace using terraform workspace select Staging.
16. Executed terraform plan to review the execution plan.
17. Executed terraform apply and successfully created the Staging EC2 instance.

Prod Environment:

18. Selected the Prod workspace using terraform workspace select Prod.
19. Executed terraform plan to review the execution plan.
20. Executed terraform apply --auto-approve and successfully created the Prod EC2 instance.

Verification:

21. Executed terraform workspace list to verify all three workspaces.
22. Selected each workspace individually using terraform workspace select.

23. Executed terraform state list in each workspace.
24. Verified that each workspace maintained its own EC2 instance in its separate Terraform state.

Project Folder Structure

```
terraform_workspace/
│
├── main.tf
├── variables.tf
│
└── terraform.tfstate.d/
    │
    ├── Dev/
    │   └── terraform.tfstate
    │
    ├── Staging/
    │   └── terraform.tfstate
    │
    └── Prod/
        └── terraform.tfstate
```

The default workspace uses the root terraform.tfstate file.

Code 1: Main Configuration

File: main.tf

```
provider "aws" {
  region = "us-east-1"
}


resource "aws_instance" "appserver" {
  ami         = var.ami[terraform.workspace]
```

```
  instance_type = "t3.micro"


  tags = {
    Name = var.tags[terraform.workspace]
  }
}
```

Code 2: Variables Configuration

File: variables.tf

```
variable "ami" {
  type = map(string)


  default = {
    Dev     = "ami-041607313c6a2a618"
    Prod    = "ami-0d97d7f07b25dd693"
    Staging = "ami-041607313c6a2a618"
  }
}


variable "tags" {
  type = map(string)


  default = {
    Dev     = "Dev-server"
    Prod    = "Prod-server"
    Staging = "Staging-server"
  }
}
```

Commands Executed

Initialize Terraform:

terraform init

Create workspaces:

terraform workspace new Dev

terraform workspace new Staging

terraform workspace new Prod

List workspaces:

terraform workspace list

Select Dev workspace and provision EC2:

terraform workspace select Dev

terraform plan

terraform apply

Select Staging workspace and provision EC2:

terraform workspace select Staging

terraform plan

terraform apply

Select Prod workspace and provision EC2:

terraform workspace select Prod

terraform plan

terraform apply --auto-approve

Verify workspace states:

terraform workspace list

terraform workspace select Dev

terraform state list

terraform workspace select Staging

terraform state list

terraform workspace select Prod

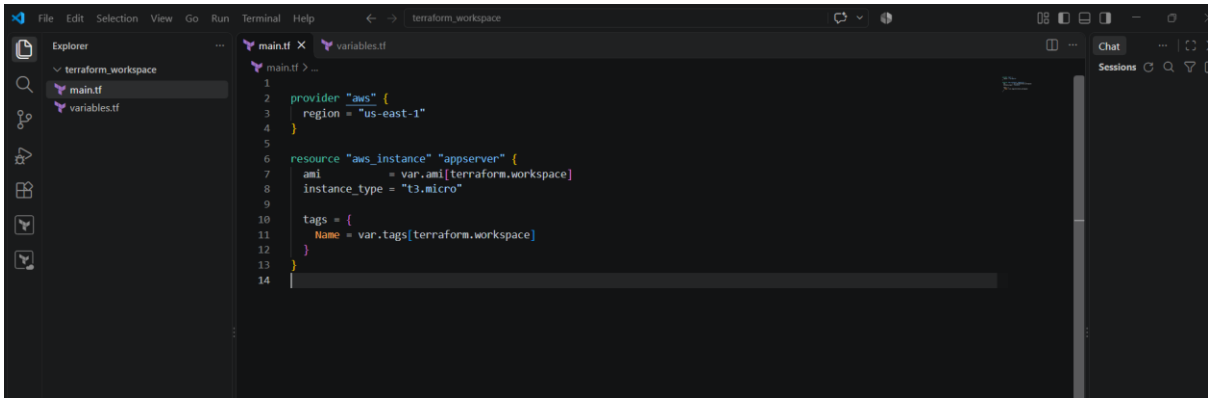
terraform state list

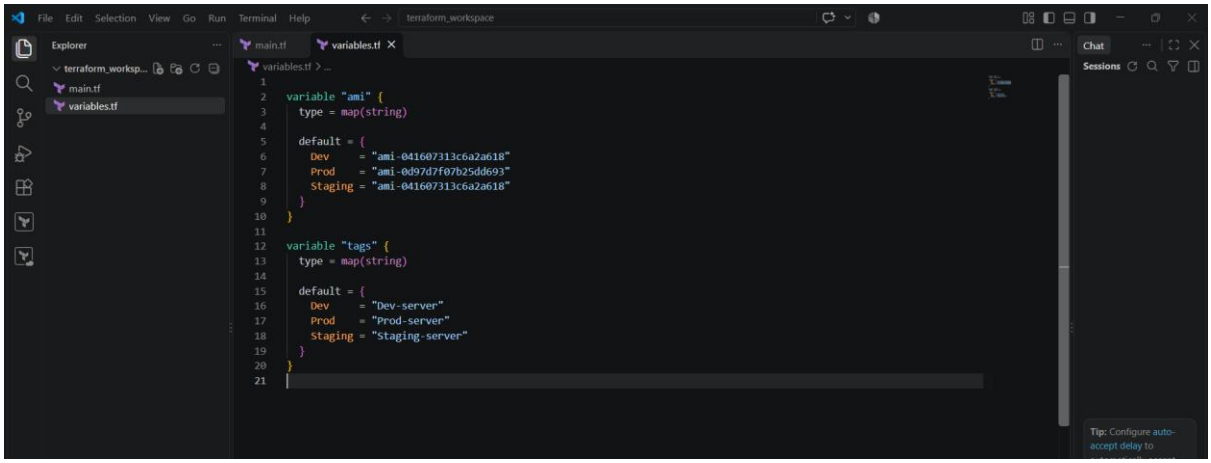
Result

Successfully created three separate EC2 instances using Terraform Workspaces.

Environment	Workspace	EC2 Instance ID
Dev	Dev	i-0cd0d1818271e8b14
Staging	Staging	i-083774aa425a57589
Prod	Prod	i-040155db4d8ae928e

Verified that each workspace maintained a separate Terraform state containing its respective EC2 instance.





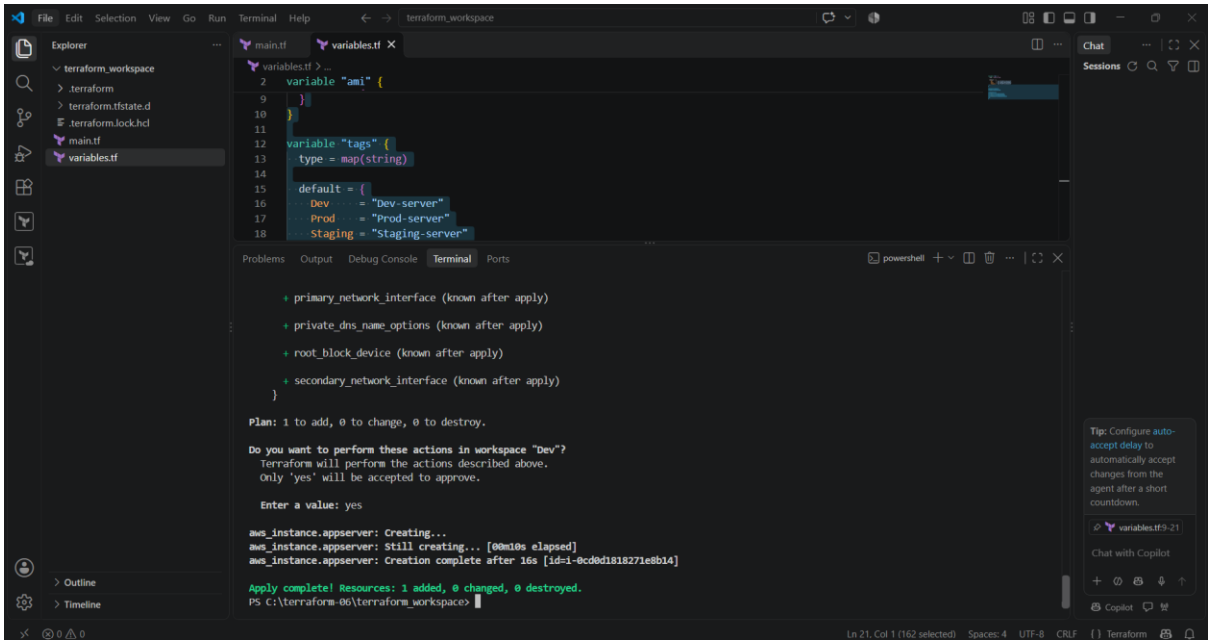
```
Problems  Output  Debug Console  Terminal  Ports                                   powershell

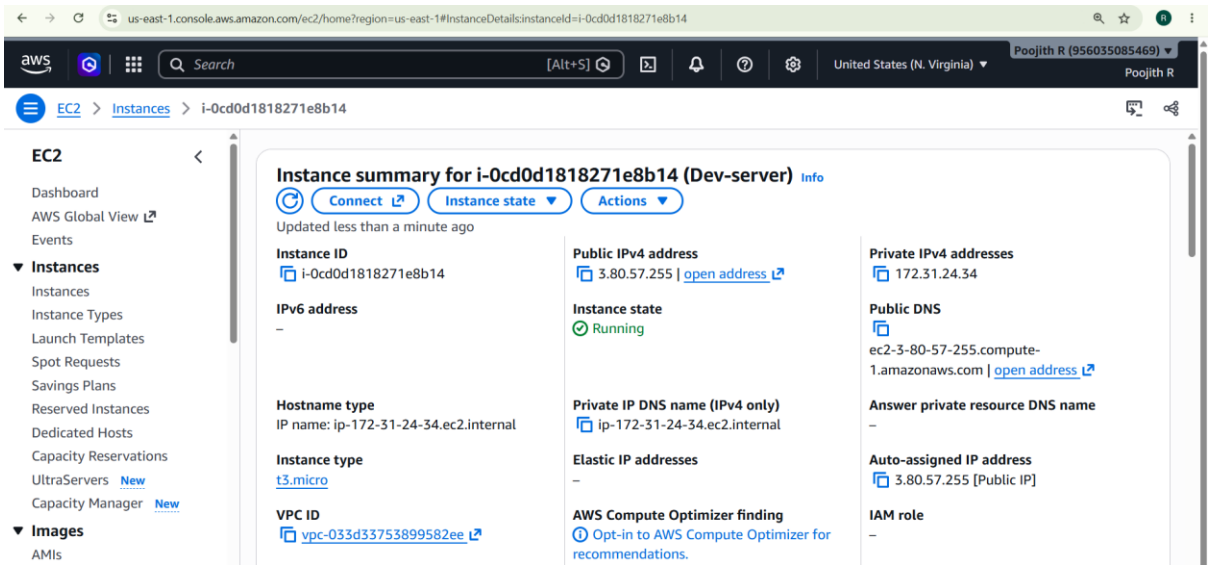
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\terraform-06\terraform_workspace> terraform workspace new Dev
Created and switched to workspace "Dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\terraform-06\terraform_workspace> terraform workspace new Staging
Created and switched to workspace "Staging"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\terraform-06\terraform_workspace> terraform workspace new Prod
Created and switched to workspace "Prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\terraform-06\terraform_workspace> terraform workspace list
  default
  Dev
* Prod
  Staging
```





```
PS C:\terraform-06\terraform_workspace> terraform workspace select Staging
Switched to workspace "Staging".
PS C:\terraform-06\terraform_workspace> terraform workspace show
Staging
PS C:\terraform-06\terraform_workspace> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.appserver will be created
  + resource "aws_instance" "appserver" {
      + ami                                  = "ami-041607313c6a2a618"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
```

```
Problems  Output  Debug Console  Terminal  Ports                                   powershell

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform
apply" now.
PS C:\terraform-06\terraform_workspace> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.appserver will be created
  + resource "aws_instance" "appserver" {
      + ami                                  = "ami-041607313c6a2a618"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
```

```
Problems  Output  Debug Console  Terminal  Ports                                   powershell

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

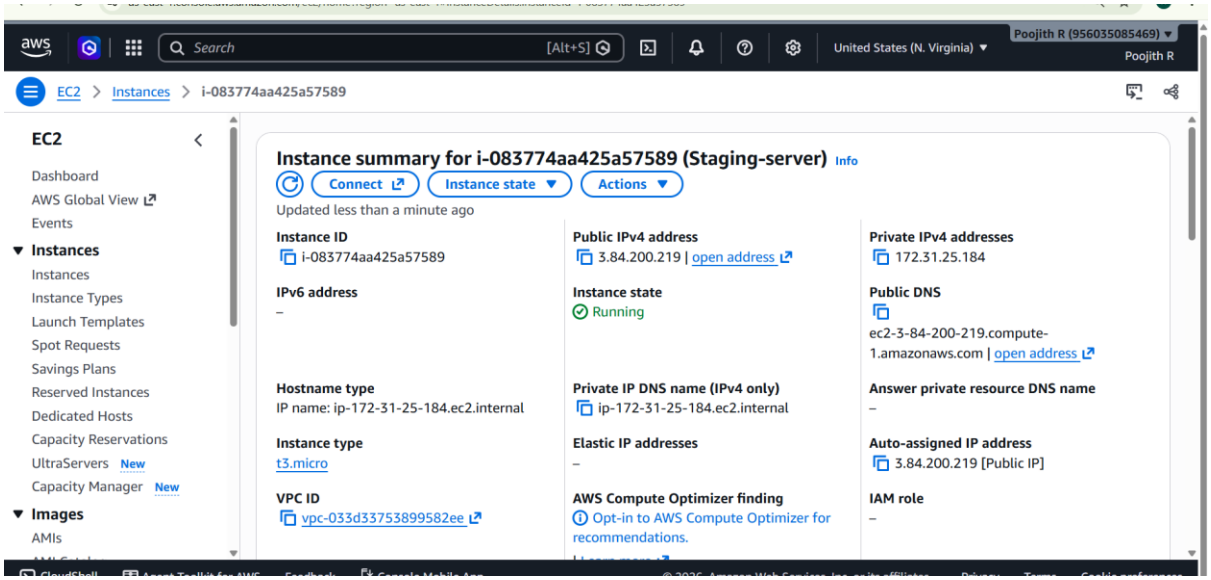
  Plan: 1 to add, 0 to change, 0 to destroy.

  Do you want to perform these actions in workspace "Staging"?
    Terraform will perform the actions described above.
    Only 'yes' will be accepted to approve.

    Enter a value: yes

  aws_instance.appserver: Creating...
  aws_instance.appserver: Still creating... [00m10s elapsed]
  aws_instance.appserver: Creation complete after 17s [id=i-083774aa425a57589]

  Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
  PS C:\terraform-06\terraform_workspace>
```




```
Problems  Output  Debug Console  Terminal  Ports                                                    terraform

PS C:\terraform-06\terraform_workspace> terraform workspace select Prod
Switched to workspace "Prod".
PS C:\terraform-06\terraform_workspace> terraform workspace show
Prod
PS C:\terraform-06\terraform_workspace> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.appserver will be created
  + resource "aws_instance" "appserver" {
      + ami                                  = "ami-0d97d7f07b25dd693"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    terraform

apply" now.
PS C:\terraform-06\terraform_workspace> terraform apply --auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.appserver will be created
  + resource "aws_instance" "appserver" {
      + ami                                  = "ami-0d97d7f07b25dd693"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
```

```
      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

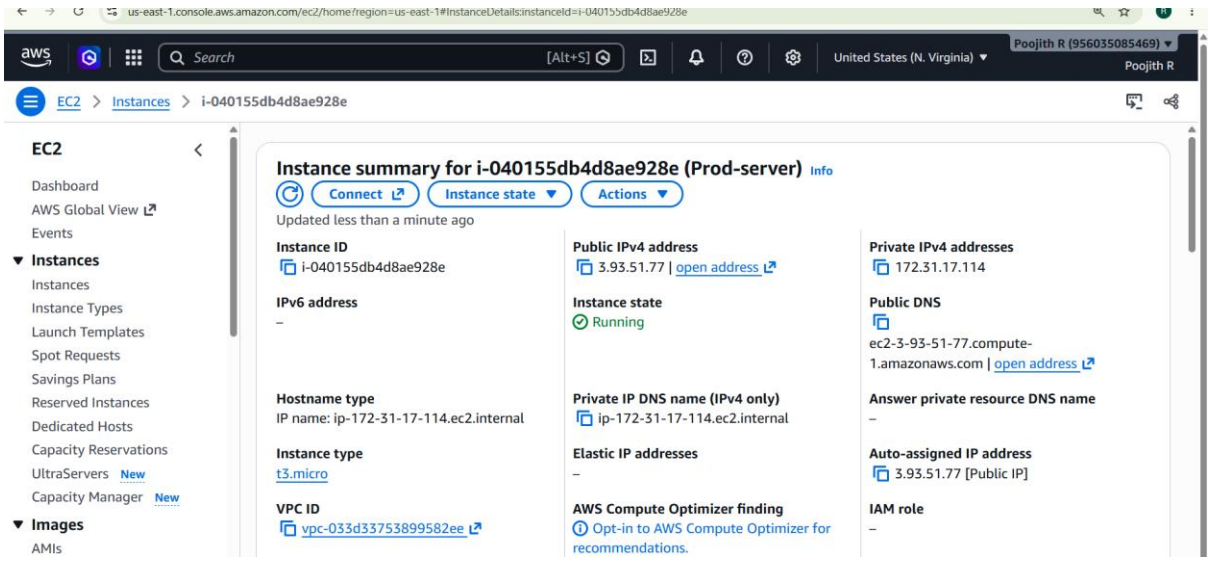
      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
aws_instance.appserver: Creating...
aws_instance.appserver: Still creating... [00m10s elapsed]
aws_instance.appserver: Creation complete after 17s [id=i-040155db4d8ae928e]

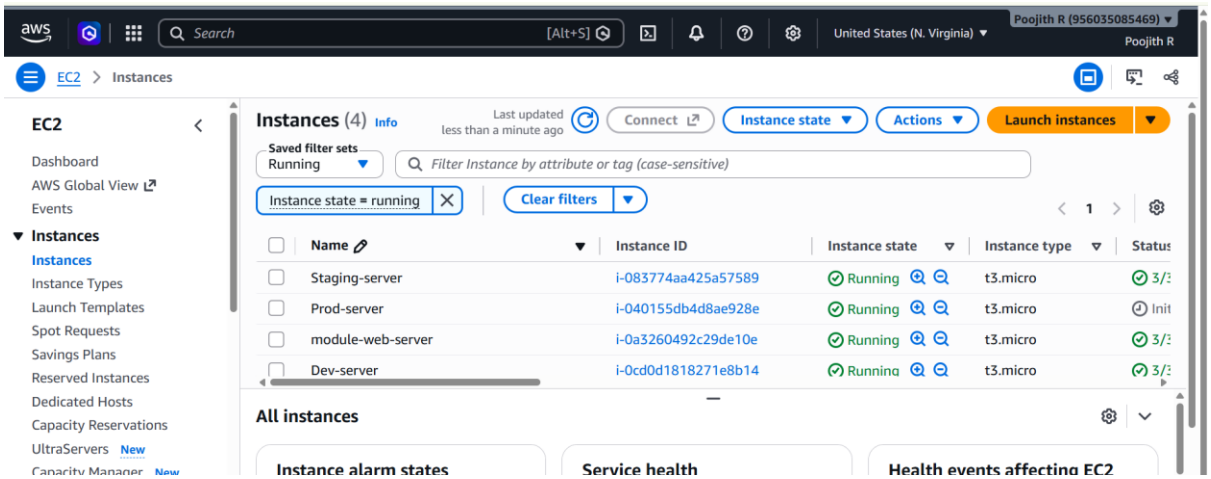
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform-06\terraform_workspace>
```




```
Problems  Output  Debug Console  Terminal  Ports                                  powershell

PS C:\terraform-06\terraform_workspace> terraform workspace list
  default
  Dev
* Prod
  Staging

PS C:\terraform-06\terraform_workspace> terraform workspace select Dev
>> terraform state list
Switched to workspace "Dev".
aws_instance.appserver
PS C:\terraform-06\terraform_workspace> terraform workspace select Staging
>> terraform state list
Switched to workspace "Staging".
aws_instance.appserver
PS C:\terraform-06\terraform_workspace> terraform workspace select Prod
>> terraform state list
Switched to workspace "Prod".
aws_instance.appserver
PS C:\terraform-06\terraform_workspace>
```


Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Instances (4) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Running
Filter Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status
Staging-server
i-083774aa425a57589
Running
t3.micro
Prod-server
i-040155db4d8ae928e
Running
t3.micro
module-web-server
i-0a3260492c29de10e
Running
t3.micro
Dev-server
i-0cd0d1818271e8b14
Running
t3.micro
All instances
Instance alarm states
Service health
Health events affecting EC2